



PostgreSQL MVCC & Vacuuming

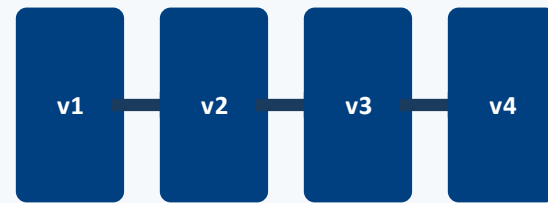
Understanding concurrency, heap storage & maintenance

Agenda

- Overview of concurrency control concepts
- Heap storage and tuples in PostgreSQL
- Vacuuming process and autovacuum
- Tuning & monitoring vacuuming
- Discussion and next steps

What is MVCC?

- MVCC = Multiversion Concurrency Control
- Readers never block writers; writers rarely block readers
- Implemented via hidden xmin/xmax columns storing transaction IDs



Version chain (newest → oldest)

Transactions & Isolation Levels

- Transaction IDs assigned when a statement begins
- Read committed is the default isolation level
- PostgreSQL aliases READ UNCOMMITTED to READ COMMITTED

Tuple Versioning

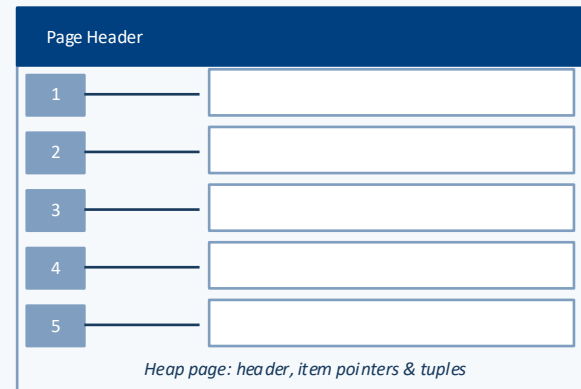
- Each row stores creation (xmin) and expiration (xmax) transaction IDs
- Deletes and updates mark rows as expired (dead tuples)
- Updates are implemented as delete + insert to preserve snapshots

Row Visibility & Transaction Space

- A row is visible if its xmin is committed and $\leq$ your transaction ID
- Rows with future xmin or uncommitted xmin are invisible
- Transaction ID space is circular ($\approx 4B$ values, half past, half future)

Heap Storage Basics

- PostgreSQL tables are heap-organized with no clustered index
- Data pages are 8 KB and appended sequentially as tables grow
- Item pointers map index entries to the latest tuple version



- Dead tuples accumulate when rows are updated or deleted
- Sequential scans must skip dead tuples, increasing I/O
- PostgreSQL may choose sequential scans for small tables

Why Vacuum?

- Reclaims space used by dead tuples so it can be reused
- Marks very old rows (and pages) as frozen to speed up scans
- Updates visibility maps and indexes to reflect deleted rows

- Clears references to expired tuples and compacts space
- Unused item pointers remain for reuse (reduces fragmentation)
- Dead tuple counts can be monitored to evaluate health

Autovacuum Overview

- Background process automatically triggers vacuum & analyze
- Uses scale factor (%) and threshold (rows) to decide when to run
- Auto analyze updates statistics for the planner

- Scale factor and threshold can be adjusted per table
- Lower values make vacuum run more often on write-heavy tables
- Maintenance work mem controls memory used for vacuum

Benefits of MVCC

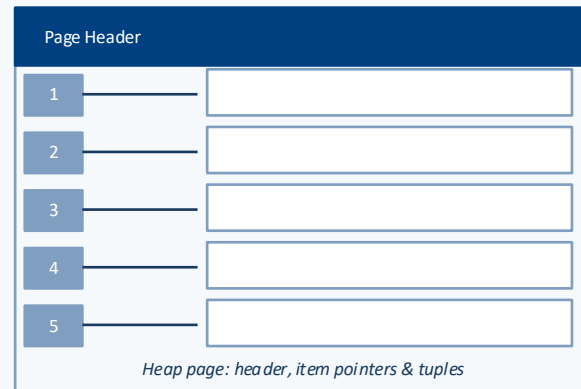
- High concurrency with minimal blocking
- Supports long-running read queries without interfering with writes
- Enables consistent snapshots for transactions

Challenges & Drawbacks

- Dead tuple accumulation increases disk usage
- Requires regular vacuuming and monitoring
- Transaction ID wraparound if vacuuming is neglected

Heap Storage Detailed

- Heap files are unordered sequences of pages containing tuples
- New rows are appended sequentially to heap pages
- Indexes point to item pointers; pointers redirect to latest tuple



- Implemented by PostgreSQL, Oracle, MySQL InnoDB and others
- Ideal for OLTP workloads with high concurrency
- Also beneficial for mixed analytical & transactional systems

- Watch dead tuple counts, `pg_stat_user_tables` and `relpages`
- Check freeze age to avoid transaction wraparound
- Use built-in views & tools (`pg_stat_activity`, autovacuum logs)

- MVCC and vacuuming are critical to PostgreSQL's longevity
- Tune autovacuum rather than disabling it
- Community resources: Bruce Momjian's talks, internals website

- Review questions on autocommit, autovacuum versions and performance
- Benchmark databases carefully: results depend on schema and workload
- Stay curious: explore official docs and community blogs

Conclusion & Next Steps

- Recap: MVCC ensures concurrency; heap storage organizes data; vacuum keeps tables healthy
- Continue learning through practice and reading
- Join upcoming webinars and engage with the PostgreSQL community